1. Problem Title & Source

Title: **Kth Largest Factor**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

You are given a number **N** and an integer **K**.

A **factor** of a number is any number that divides it exactly.

Your task is to determine the **Kth largest factor of N**.

Rules:

- List all factors of **N**
- Sort them in **descending order**
- Return the **Kth largest factor**

If the number of factors is **less than K**, return:

1

3. Examples

Example 1

Input

12

3

Output

4

Explanation

Factors of 12

1, 2, 3, 4, 6, 12

Sorted descending

12, 6, 4, 3, 2, 1

3rd largest factor

4

Example 2

Input

30

4

Output

5

Explanation

Factors of 30

1, 2, 3, 5, 6, 10, 15, 30

Descending order

30, 15, 10, 5, 6, 3, 2, 1

4th largest factor

5

Example 3

Input

7

3

Output

1

Explanation

Factors of 7

1, 7

Only 2 factors exist.

Since **K = 3**, output

1

4. Constraints

1 ≤ N ≤ 10^6

1 ≤ K ≤ 100

Observations

- Number of factors is limited
- We should avoid checking all numbers up to **N**

5. Core Concept (Pattern)

Factor Finding + Number Theory

Key idea:

Factors come in pairs

Example

12

Pairs

1 × 12

2 × 6

3 × 4

6. Thought Process

Brute Force Idea

Check every number from

1 → N

If divisible

N % i == 0

store factor.

Time complexity

O(N)

Too slow for large N.

Optimized Idea

We only need to check up to

√N

Because factors appear in pairs.

Example

N = 36

Check until

√36 = 6

7. Visualization

Example

N = 12

Check numbers

i	Factor Pair
1	112
2	2,6
3	3,4

Factors

1,2,3,4,6,12

Descending

12,6,4,3,2,1

3rd largest

4

8. Pseudocode

```
read N
read K

create empty list factors

for i from 1 to √N

   if N % i == 0

      add i to list

      if i != N/i
         add N/i

sort factors in descending order

if K ≤ size of factors
   print factors[K-1]
else
   print 1
```

9. Code Implementation

Python

```
import math

n = int(input())
k = int(input())

factors = []

for i in range(1, int(math.sqrt(n)) + 1):

   if n % i == 0:

      factors.append(i)
```

```
    if i != n // i:
        factors.append(n // i)

factors.sort(reverse=True)

if k <= len(factors):
    print(factors[k-1])
else:
    print(1)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();

        List<Integer> factors = new ArrayList<>();

        for(int i=1;i*i<=n;i++){

            if(n%i==0){

                factors.add(i);

                if(i != n/i)
                    factors.add(n/i);
            }
        }

        Collections.sort(factors, Collections.reverseOrder());
```

```
        if(k <= factors.size())
            System.out.println(factors.get(k-1));
        else
            System.out.println(1);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(√N)

Because we only iterate until square root.

Space Complexity

O(F)

Where

F = number of factors

11. Common Mistakes

1. Checking factors up to **N**
2. Forgetting paired factors
3. Incorrect sorting
4. Missing case when **K > number of factors**

12. Detailed Dry Run

Input

N = 12

K = 3

Factors found

1,12

2,6

3,4

All factors

1,2,3,4,6,12

Sorted descending

12,6,4,3,2,1

3rd largest

4

13. Real-Life Applications

Factor-based problems appear in:

- Cryptography
- Number theory
- Resource distribution problems
- Mathematical optimization

14. Common Traps

1. Missing i != n/i condition
2. Forgetting to sort descending
3. Off-by-one indexing
4. Handling perfect squares incorrectly

15. Related Problems

Number theory problems:

LC 507 Perfect Number

LC 204 Count Primes

LC 2520 Count Digits That Divide Number

16. Alternate Approaches

Approach	Complexity	Notes
Check up to N	O(N)	slow
Check up to √N	O(√N)	optimal

17. Why This Solution Works

Since factors always appear in **pairs**, checking only up to **√N** guarantees we discover all factors efficiently.

This reduces the complexity dramatically compared to checking all numbers.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Find **Kth smallest factor**
2. Count **total factors**
3. Check if number is **perfect number**
4. Find **largest proper divisor**